

PostalMS

Time Complexity Analysis

Custom Hash Table | Custom BST | Custom Queue | Business Logic Algorithms

1. Introduction

This document provides a detailed time complexity analysis of all three custom data structures and the five new business logic algorithms added in Sprint 4. All data structures were built from scratch without using any standard template library collections.

1.1 Big O Notation Key

Notation	Meaning
$O(1)$	Constant time. The operation takes the same time regardless of input size. Best possible complexity.
$O(\log n)$	Logarithmic time. Time grows slowly. Each step eliminates half the remaining options.
$O(n)$	Linear time. Time grows proportionally with input size. Acceptable for full traversals.

2. Custom Hash Table

2.1 Overview

The CustomHashTable uses a polynomial rolling hash to convert tracking IDs to array indices. $O(1)$ average for all core operations. Auto-resize at 75% load factor.

2.2 Worst Case

Worst case $O(n)$ occurs when all keys hash to the same slot creating a full collision chain. Extremely unlikely with a good hash function and auto-resize keeping load factor below 0.75.

2.3 Time Complexity Table

Operation	Best Case	Average	Worst Case	Reason for Worst Case
Add	$O(1)$	$O(1)$	$O(n)$	All keys hash to same slot -- full chain traversal required
Search	$O(1)$	$O(1)$	$O(n)$	Target key at end of long collision chain
Delete	$O(1)$	$O(1)$	$O(n)$	Target key at end of long collision chain
ContainsKey	$O(1)$	$O(1)$	$O(n)$	Calls Search internally -- same complexity
Resize	$O(n)$	$O(n)$	$O(n)$	Every item must be re-hashed into the new table
Clear	$O(n)$	$O(n)$	$O(n)$	Every slot in the array must be set to null

3. Custom Binary Search Tree

3.1 Overview

The CustomBST uses the BST ordering property to eliminate half the remaining nodes at each step. $O(\log n)$ average insert and search. In-order traversal always $O(n)$.

3.2 $O(\log n)$ in Practice

Parcels	$O(n)$ Linear Search	$O(\log n)$ BST Search
1,000 parcels	Up to 1,000 comparisons	Up to 10 comparisons
10,000 parcels	Up to 10,000 comparisons	Up to 14 comparisons
1,000,000 parcels	Up to 1,000,000 comparisons	Up to 20 comparisons

3.3 Time Complexity Table

Operation	Best Case	Average	Worst Case	Reason for Worst Case
Insert	$O(\log n)$	$O(\log n)$	$O(n)$	Sorted input creates degenerate tree (linked list shape)
Search	$O(1)$	$O(\log n)$	$O(n)$	Target at deepest leaf of degenerate tree
Delete	$O(\log n)$	$O(\log n)$	$O(n)$	Target at deepest leaf of degenerate tree
Contains	$O(1)$	$O(\log n)$	$O(n)$	Calls Search internally -- same complexity
InOrder	$O(n)$	$O(n)$	$O(n)$	Must visit every single node exactly once
FindMin	$O(1)$	$O(\log n)$	$O(n)$	Follows left pointers to deepest left leaf
Clear	$O(1)$	$O(1)$	$O(1)$	Sets root to null -- garbage collection handles rest

4. Custom Queue

4.1 Overview

The CustomQueue is a linked list with front and rear pointers. $O(1)$ for all core operations with no worst-case degradation -- the most predictable structure in PostalMS.

4.2 Time Complexity Table

Operation	Best Case	Average	Worst Case	Reason for Worst Case
Enqueue	$O(1)$	$O(1)$	$O(1)$	Rear pointer gives direct access -- no traversal
Dequeue	$O(1)$	$O(1)$	$O(1)$	Front pointer gives direct access -- no traversal
Peek	$O(1)$	$O(1)$	$O(1)$	Front pointer -- just read the value
IsEmpty	$O(1)$	$O(1)$	$O(1)$	Single null check on front pointer
Count	$O(1)$	$O(1)$	$O(1)$	Maintained as a field -- incremented and decremented directly
ToList	$O(n)$	$O(n)$	$O(n)$	Must traverse every node front to rear
Clear	$O(1)$	$O(1)$	$O(1)$	Sets front and rear to null and count to 0

5. Business Logic Algorithms (New -- Sprint 4)

5.1 Gmail Validation -- $O(n)$

The validation checks whether an email ends with @gmail.com. EndsWith scans the string of length n . Since emails are always under 100 characters this is effectively $O(1)$ in practice.

Operation	Best Case	Average	Worst Case	Reason for Worst Case
ValidateGmail	$O(1)$	$O(n)$	$O(n)$	EndsWith and StartsWith scan the email string of length n

5.2 Stamp Price Calculation -- $O(1)$

Fixed arithmetic operations regardless of quantity ordered. Multiply, add, round -- all constant time.

Operation	Best Case	Average	Worst Case	Reason for Worst Case
CalcStampTotal	$O(1)$	$O(1)$	$O(1)$	Fixed number of arithmetic operations regardless of quantity

5.3 City Detection -- $O(c)$

Searches through $c = 7$ supported cities to find a match. Since c is always 7 this is effectively $O(1)$ constant time in practice.

Operation	Best Case	Average	Worst Case	Reason for Worst Case
GetUserCity	$O(1)$	$O(c)$	$O(c)$	Scans c supported cities to find match -- c is always 7
ShowLocationsForCity	$O(1)$	$O(L)$	$O(L)$	Renders L location cards for selected city

5.4 Stamp Order Flow -- $O(1)$

Fixed validation steps plus one list add. All validation is constant time. Adding to a list is $O(1)$ amortized.

Operation	Best Case	Average	Worst Case	Reason for Worst Case
PlaceStampOrder	$O(1)$	$O(1)$	$O(1)$	Fixed validation steps plus $O(1)$ list add -- amortized
CheckMissedPickup	$O(1)$	$O(1)$	$O(1)$	Single date comparison -- no loops or searches

5.5 Receiver Details Validation -- $O(1)$

Validates each receiver field in Step 4 of the ParcelsView send form. Each field check is a single string comparison or null check. The total number of fields is always fixed at 6 required fields plus 1 optional field. All checks are $O(1)$.

Operation	Best Case	Average	Worst Case	Reason for Worst Case
ValidateReceiverDetails	$O(1)$	$O(1)$	$O(1)$	Fixed number of field checks -- always 6 required fields

6. Overall Comparison

#	Structure or Algorithm	Best	Average	Worst	Notes
1	Hash Table Add/Search/Delete	$O(1)$	$O(1)$	$O(n)$	Worst case extremely rare
2	Hash Table Resize	$O(n)$	$O(n)$	$O(n)$	Amortized $O(1)$ per insertion
3	BST Insert/Search/Delete	$O(1)$	$O(\log n)$	$O(n)$	$O(n)$ only with sorted input
4	BST InOrder Traversal	$O(n)$	$O(n)$	$O(n)$	Always visits all n nodes
5	Queue Enqueue/Dequeue/Peek	$O(1)$	$O(1)$	$O(1)$	Always constant -- no degradation
6	Queue ToList	$O(n)$	$O(n)$	$O(n)$	Must visit all n queue items
7	Gmail Validation	$O(1)$	$O(n)$	$O(n)$	n = email length, always < 100
8	Stamp Price Calculation	$O(1)$	$O(1)$	$O(1)$	Fixed arithmetic only
9	City Detection	$O(1)$	$O(c)$	$O(c)$	$c = 7$ cities, effectively $O(1)$
10	Stamp Order Flow	$O(1)$	$O(1)$	$O(1)$	Amortized constant
11	Receiver Details Validation	$O(1)$	$O(1)$	$O(1)$	Fixed 6 field checks

7. Conclusion

The three custom data structures were chosen for their time complexity characteristics. Hash Table $O(1)$ for instant parcel lookup, BST $O(\log n)$ for sorted display and $O(n)$ in-order traversal, Queue $O(1)$ for FIFO delivery processing. The five new Sprint 4 algorithms are all effectively constant time in practice due to bounded input sizes -- email length under 100 characters, exactly 7 cities, exactly 6 receiver fields.